

Web программирование

Курс для бакалавриата

Чернышев Вячеслав

Кэширование

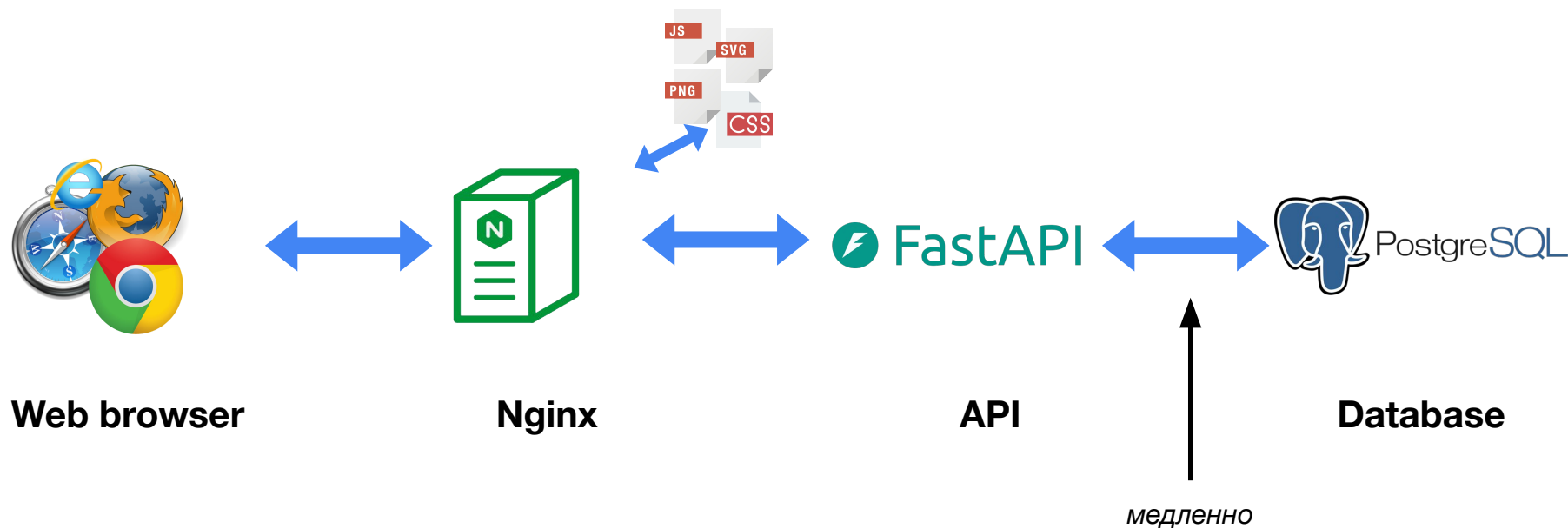
Содержание лекции

1. Кэширование
2. Кэш для FastAPI
3. Redis
4. Паттерны

Что такое кэш

это высокоскоростной уровень хранения, на котором требуемый набор данных, как правило, временного характера.

Архитектура Веб приложение на текущий момент



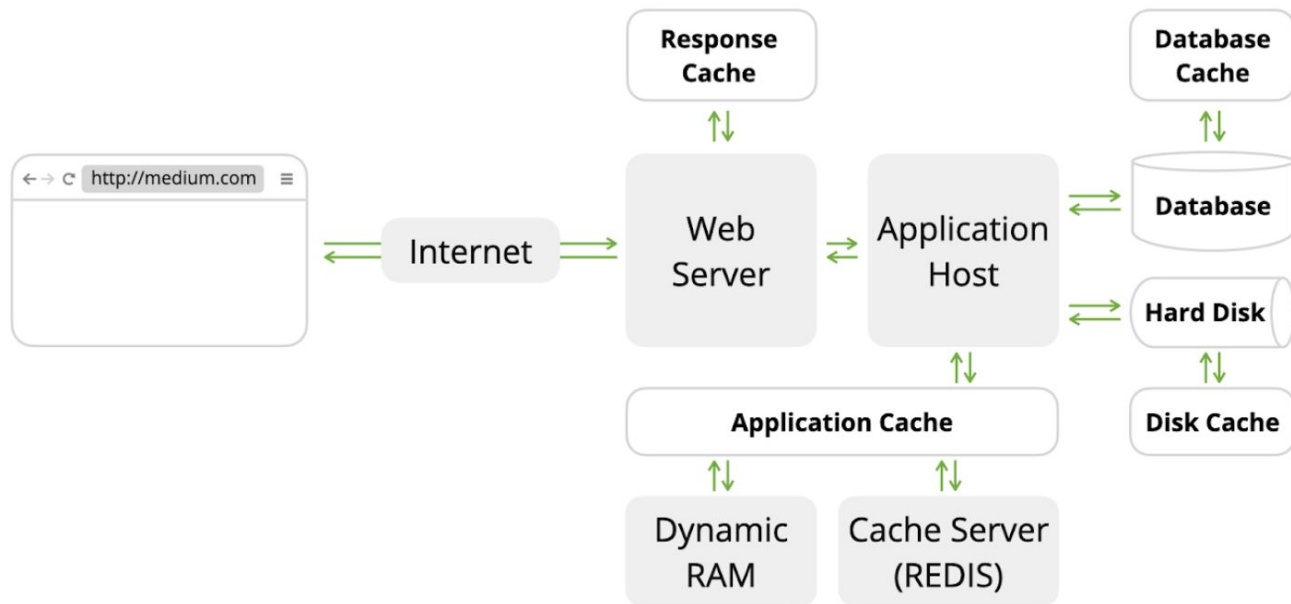
Проблемы производительности

- Одинаковые запросы к серверу, которые требуют различных вычислений
- Тяжелый запрос к БД
- Запрос к стороннему сервису

Виды кэша

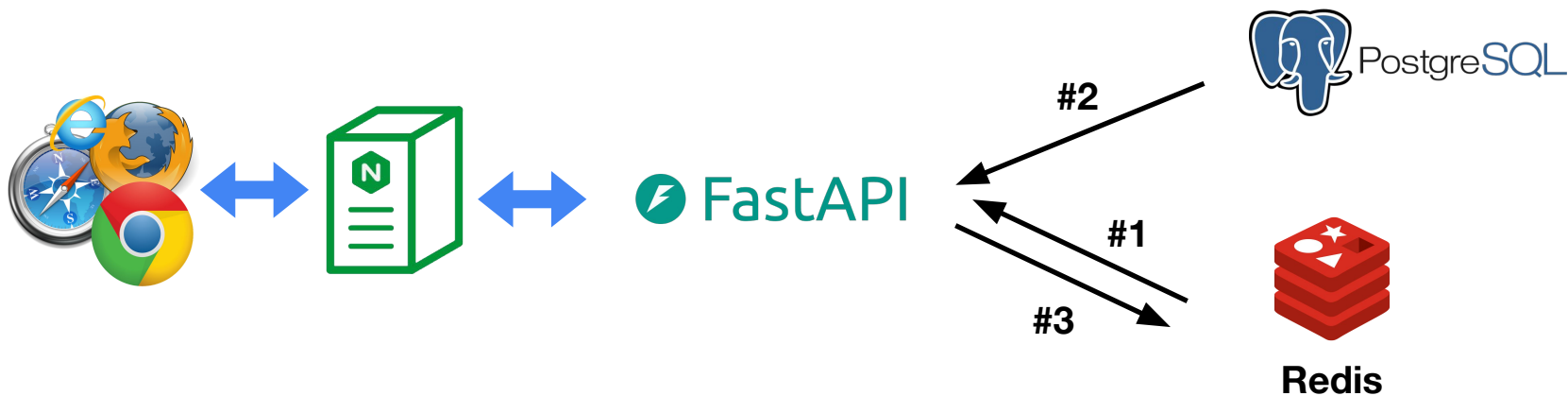
- Client-side (На уровне браузера)
- DNS
- Кэш на уровне Proxy, CDN
- Кэш на уровне приложения
- Кэш на уровне базы данных (буфер баз данных)

Виды кэша



Зачем нужен кэш в Веб

- Ускорение доставки
- Уменьшение нагрузки



Где может храниться кэш

- Оперативная память сервера
- Локально на диске
- Отдельно выделенный кэширующий сервер

Другие примеры использования кэша

- Несколько сервисам нужно общее быстрое хранилище
- Сбор аналитики по запросам

HDD vs RAM

- HDD медленнее по сравнению с оперативной памятью
- HDD нужен для хранения больших объемов данных на протяжении долгого времени
- RAM нужен для хранения относительно небольшого количества данных на короткий промежуток времени

Как кэшировать в FastAPI приложении

Хранить в оперативной памяти приложения

```
class InMemoryCache:
    def __init__(self): ...

    def set(self, key: str, value: Any, ttl: int = 300) -> None: ...
    def get(self, key: str) -> Any: ...
    def delete(self, key: str) -> bool: ...
    def exists(self, key: str) -> bool: ...
    def clear(self) -> None: ...
    def cleanup(self) -> int: ...
    def size(self) -> int: ...
    def get_ttl(self, key: str) -> Optional[float]: ...

cache = InMemoryCache()

cache.set("user:1", {"name": "Alice", "age": 30}, ttl=10)
cache.set("user:2", {"name": "Bob", "age": 25}, ttl=5)
cache.set("session:abc123", "active", ttl=3)
```

1. Сверхгорячие справочники
2. Конфиги, флаги, которые редко меняются
3. Предвычисленные значения для старта

Хранение в оперативной памяти приложения



- Скорость. Нет сетевых операций
- Минимальная операционная сложность



- Не разделяется между инстансами
- Сбрасывается при рестарте
- Нужно вручную ограничивать по размеру и TTL

Кэш на уровне ORM

SQLAlchemy:

- Baked queries (кэширование составления sql запроса)
- First-level cache (создание python объектов в рамках сессии)



Кэш на уровне БД (Материализованные представления)

специальная таблица, которая хранит результат выполнения запроса

```
CREATE MATERIALIZED VIEW user_stats AS
SELECT
  u.id,
  u.country,
  COUNT(o.id) AS orders_count,
  SUM(o.amount) AS total_amount
FROM
  users u
  LEFT JOIN orders o ON o.user_id = u.id
GROUP BY
  u.id,
  u.country;
```

1. Повторяющиеся одинаковые запросы
2. Сложные агрегаты
3. Отчеты

```
SELECT * FROM user_stats;
```

```
REFRESH MATERIALIZED VIEW user_stats;
```

Кэш на уровне БД (Материализованные представления)



- Прозрачно для приложения
- Проще поддерживать согласованность

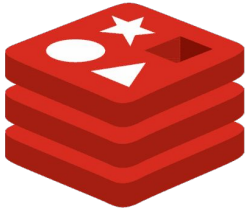


- Тяжелее отлаживать
- Не всегда дает нужный прирост

ДИСКОВЫЙ КЭШ

- Локальный файл
- SQLite
- Переживает рестарты процесса
- Медленнее памяти, Redis
- Труднее синхронизировать между инстансами

Подходит для каких-нибудь артефактов



Redis Cache

Что это

высокопроизводительное хранилище данных с открытым исходным кодом в памяти, работающее по принципу «ключ-значение» и относящееся к классу нереляционных (NoSQL) баз данных



Преимущества Redis

- In-memory
- On-disk persistence
- Replication, transactions, high availability
- Поддержка различных структур данных

Когда использовать:

- Данные редко меняются, но к ним часто обращаются
- Не относятся к важным, которые часто меняются
- Отправка и доставка сообщений
- Получения статистика
- Подготовка простейшей аналитики

Simple Database

- Единичная установка
- Низкая надежность

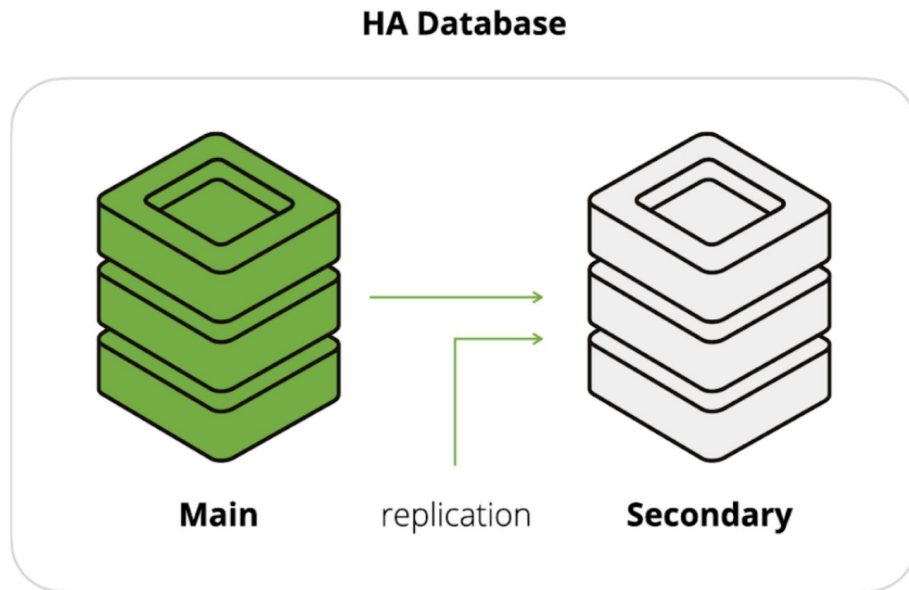
Simple Database



Main

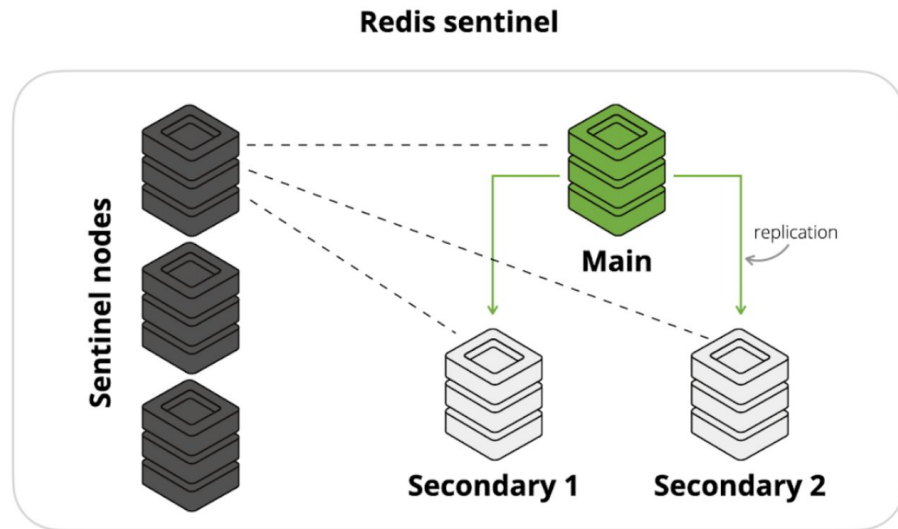
High Available

- Есть ведущий и ведомые узлы
- Основная нагрузка на ведущий экземпляр
- Ведущий отвечает за синхронизацию его с ведомыми
- Ведомый может заменить ведущий

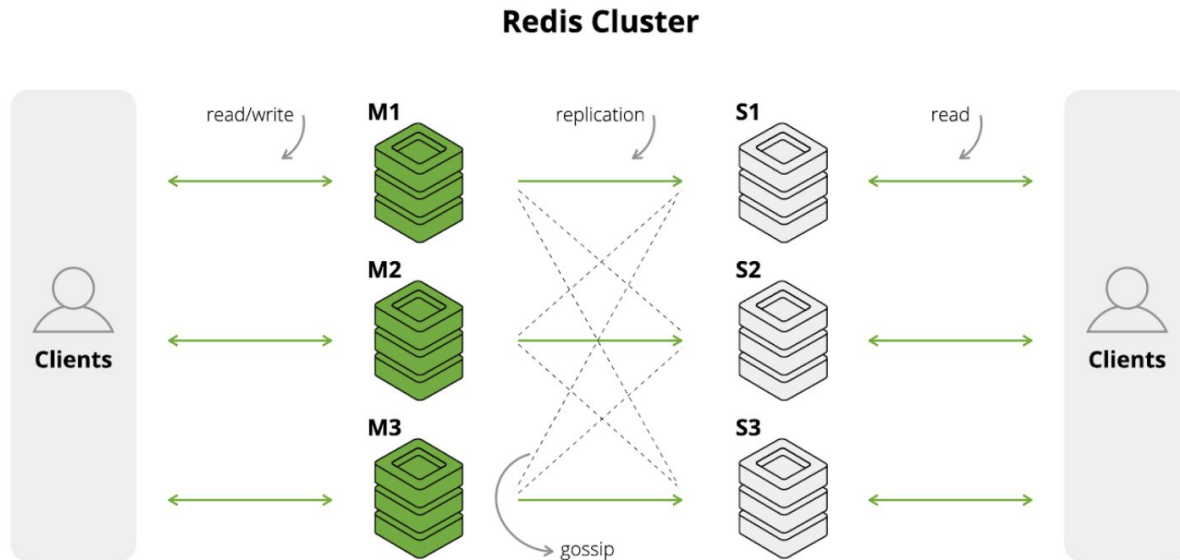


Redis sentinel

- Сервис мониторинга
- Формирование уведомлений о проблемах
- Восстановление работоспособности упавших узлов
- Предоставление информации о том, какой узел сейчас ведущий



Redis Cluster



- Распределение данных по разным серверам (шардинг)
- Протокол Gossip позволяет переключать ведущие узлы

Ключи и типы данных

- Ключ всегда строка
- Ключ не должен быть слишком длинным
- Различные сущности, входящие в название ключа следует разделять двоеточием, в то время как названия, состоящие из нескольких слов следует разделять дефисами или точками.
- Ограничение по размеру ключа: 512 Mb.

Пример нейминга: items:205:non-weighting.

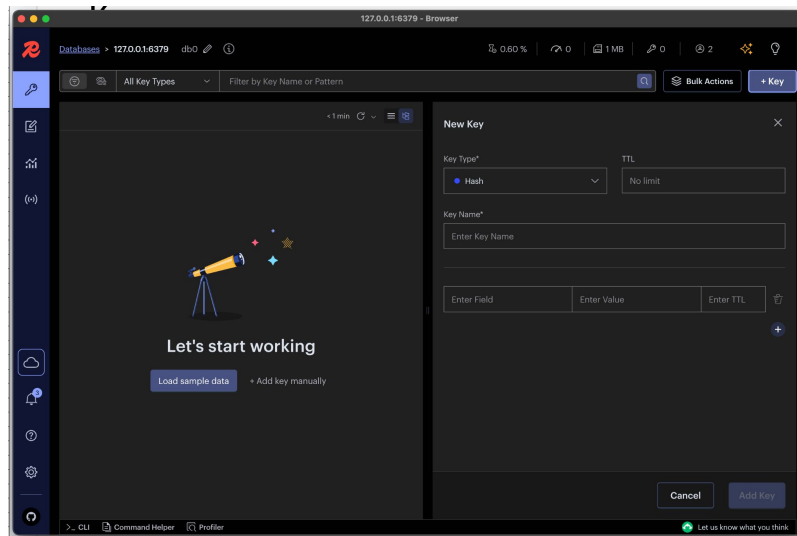
Какие значения могут принимать ключи

- Строки
- Списки (в Redis используются связанные списки)
- Хэши
- Множества (неупорядоченные наборы строк)
- Упорядоченные множества
- HyperLogLogs - для подсчета уникальных элементов

Как использовать

```
docker-compose.yml

1  version: '3.8'
2  services:
3    redis:
4      image: redis:latest
5      ports:
6        - "6379:6379"
7      volumes:
8        - ./redis_data:/data
9
```



<https://redis.io/insight/>

Команды

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Установить значение ключа	set <key> <value>	set user:1:string "my binary safe string"
Прочитать значение ключа	get <key>	get user:1:string
Вывести тип ключа	type <key>	type user:1:string
Установить значение ключа	set <key> <value>	set user:l:vaue "Alex"
Переименовать ключ	rename <key> <new-key>	rename user:1:vlaue test:1:name
Проверить существование ключа	exists <key>	exists user:l:value
Вывести все ключи по маске	keys <mask>	keys user:1:*
Удалить ключ	del <key>	del user:l:value

ttl

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Вывод срока жизни ключа	ttd <key>	ttd user:1:string
Установить срок жизни ключа в секундах	expire <key> <seconds>	expire user:1:string 6000

Команды для работы со строками

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Установить значение ключа	set <key> <value>	set user:l:greetings "hello"
Дополнить значение ключа	append <key> <value>	append user:1:greetings "world"
Вывести значение ключа	get <key>	get user:l:greeting
Вывести длину значения ключа	strlen <key>	strlen user:l:string
Вывести подстроку по индексам из ключа	getrange <key> <start index> <end index>	getrange user:l:string 6 10
Изменить подстроку по индексам значений ключа	setrange <key> <index>	setrange user:1:name 5 "Ivanov"
Вывести значение ключа	get <key>	get user:l: name

Команды для работы над числами

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Установить значение ключа	set <key> <value>	set user:1:age 25
Инкрементировать значение ключа на 1	incr <key>	incr user:1:age
Декрементировать значение ключа на 1	decr <key>	decr user:1:age
Инкрементировать на произвольное значение	incr <key> <value>	incr user:1:age 5
Декрементировать на произвольное значение	decr <key> <value>	decr user:1:age 5

Команды для работы со списками

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Добавить элемент в список	rpush <key> <item>	rush user:1:messages "Hello, world!"
Вывести элементы по индексам из списка	lrange <key> <index_args>	lrange user:1:messages 0 1
Вывести длину списка	llen <key>	llen user:1:messages
Достать следующий элемент списка	lpop <key>	lpop user:1:messages

Команды для работы над множествами

ДЕЙСТВИЕ	КОМАНДА	ПРИМЕР
Добавить элемент в список	sadd <key> <item>	sadd user:1:items "banana"
Вывести элементы по индексам из списка	scard <key>	scard user:1:items
Вывести разность множеств	sdiff <key1> <key2>	sdiff user:1:items user:2:items
Вывести количество общих элементов множеств	sdiffstore <keys>	sdiffstore user:1:items user:2:items user:3:items
Вывести общие элементы множеств	sinter <key1> <key2>	sinter user:1:items user:2:items
Проверить, входит ли в множество элемент	sismember <key> <item>	sismember user:1:items "banana"
Вывести все элементы множества	smember <key>	smembers user:1:items
Перенести элемент множества	smove <key1> <key2> <item>	smove user:1:items user:2:items "apple"
Удалить и вывести элемент множества	spop <key>	spop user:1:items
Вывести произвольный элемент множества	srandmember <key>	srandmember user:1:items

Инвалидация тегов

способ управлять кэшем не по одиночным ключам, а по логическим группам данных.

```
> set item:1 "aaa"
"OK"

> set item:2 "bbb"
"OK"

> set item:3 "ccc"
"OK"

> SADD tag:category:42 item:1 item:2
(integer) 2

> SADD tag:category:17 item:3
(integer) 1

> SMEMBERS tag:category:42
1) "item:1"
2) "item:2"
```

Версионирование тегов

```
> SET tag:category:42:version 12  
"OK"
```

```
> SET item:1:v12 "data"  
"OK"
```

```
> INCR tag:category:42:version  
(integer) 13
```

FastAPI + Redis. Init

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse
import redis.asyncio as redis

app = FastAPI()

@app.on_event("startup")
async def startup_event():
    global redis_client
    redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

@app.on_event("shutdown")
async def shutdown_event():
    global redis_client
    await redis_client.aclose()
```

FastAPI + Redis. Использование

```
@app.get("/items/{item_id}")
async def read_item(item_id: int):
    cached_item = await redis_client.get(f"item:{item_id}")
    if cached_item:
        return JSONResponse(content={"item_id": item_id, "data": cached_item, "cached": True})

    data = f"This is item {item_id}"

    await redis_client.set(f"item:{item_id}", data, ex=60)

    return JSONResponse(content={"item_id": item_id, "data": data, "cached": False})
```

Паттерны. Cache-aside (a.k.a. lazy)

1. Проверяем кэш
2. Если данных нет - читаем из источника
3. Сохраняем в кэш
4. Возвращаем данные

Паттерны. Read-through

Чтение через кэш-библиотеку

Сама библиотека обращается к источнику или кэшу

Паттерны. Write-through

При записи в БД сразу пишем в кэш

Паттерны. Write-around

При записи в БД сразу в кэш не пишем, пишем только при чтении

Паттерны. Write-back

Записываем сразу в кэш, в БД сохраняем в фоне

Rate-limits

- защита от бурстов/скрейпинга/братафорса;
- справедливое распределение квоты между пользователями/ключами API;
- дешёвое «предохранение» БД и внешних API.

Redis-auth

- Хранение refresh токенов в redis
- Хранения blacklist`а токенов в redis

Coding samples

<https://github.com/itmo-webdev/webdev-2025>



Вопросы?